

# Design of Efficient Cyclic Redundancy Check-32 using FPGA

Mohamed S. Abdulnabi  
Computer and Network Engineering  
Sudan University of Science and Technology  
Khartoum, Sudan  
mohd.salah181@gmail.com

Hisham Ahmed (IEEE Member)  
Assistant Professor, Electronics Engineering  
Sudan University of Science and Technology  
Khartoum, Sudan  
hisham@iccee.org

**Abstract**—In Data Integrity, Cyclic Redundancy Check (CRC) is error detection technique. CRC treat a group of binary bits of a message by dividing it by a fixed binary number, the resulted remainder is the checksum that will be attached to the message. On the receiver side the same division could be performed and the receiver can compare the remainder with the transmitted checksum. This paper explains the design of CRC32 that used in Ethernet using Field Programmable Gate Array (FPGA) Virtex-7. The design built based on lookup tables and slicing-by-16 algorithm that performed the calculations of the CRC32 in parallel. Xilinx ISE used as IDE and I-Sim used for the simulation. The resulted processing time is equal to 1.250 ns with low power consumption and low device utilization.

**Keywords**—CRC32, FPGA, Xilinx

## I. INTRODUCTION

The internet and data communication considered one of the most powerful and important creations in all of human history. The internet already has had its impact on science, government, business, communication and humanity. The advances in telecommunications technologies and services facilitated the massive exchange of data between client devices, data center and the clouds. novel applications are widely found that span across diverse fields as never before. Nevertheless, the Internet of Things (IoT) is a recent communication paradigm that envisions a near future, in which the objects of everyday life will be equipped with microcontrollers, transceivers for digital communication, and suitable protocol stacks that will make them able to communicate with one another and with users, so the IoT is becoming an integral part of the Internet [1]. Total internet traffic has experienced dramatic growth in the past two decades. More than 20 years ago, in 1992, global internet networks carried approximately 100 GB of traffic per day. Ten years later, in 2002, global internet traffic amounted to 100 GB per seconds (GBps). In 2015, global internet traffic reached more than 20000 GBps [1]. By 2020 global IP traffic will reach 2.3 Zettabyte per year (Zettabyte is  $10^{21}$  bytes or 1 trillion Gigabytes), or 194 Exabyte (Exabytes is  $10^{18}$  or 1 billion Gigabytes) per month. The global fixed broadband speed will reach 47.7 Mbps by 2020. Not yet, by 2020 every minute a million minutes of video content will cross the network and the amount of video on demand (VoD) will be equivalent to 7.2

Million DVD per month. Mobile data traffic will reach 30.6 Exabytes per month by 2020. Globally, IP traffic will reach 511 terabit per seconds (Tbps) in 2020, this is the equivalent of 142 million people streaming high-definition (HD) video simultaneously, all day, every day [1].

The CRC32 (Cyclic Redundancy Check 32) which is the error detection technique that used in the Ethernet will be affected by the huge increase of the data traffic. The existing throughput of the CRC32 in the backbone and core devices cannot handle the future traffic, accordingly the throughput of the CRC32 should be increased. This paper explains the design of CRC32 that used in Ethernet using Field Programmable Gate Array (FPGA) Virtex-7. The design built based on lookup tables and slicing-by-16 algorithm that perform the calculation of the CRC32 in parallel.

The computation of the CRC using lookup tables explained by Sarwate, he proved that finding out the checksum value using lookup table method is more efficient than other traditional methods such as Linear Feedback Shift Register (LFSR) which treat the data serially [2]. Anand and Bajarangbali in [3] implement the CRC algorithm on the FPGA by using reduced lookup tables with slicing-by-16 and they achieved a throughput reached up to 40 Gbps. Another studies in [4], [5] and [6] discuss the implementation of the CRC in parallel.

Rest of paper is described as follows: section II describes briefly the CRC algorithm, section III describe proposed work by the author. Results and conclusion are given in sections IV & V.

## II. CRC ALGORITHM

Cyclic redundancy checks (CRC) is used for detecting the corruption of digital data during the transmission, processing or the storing of raw data. The CRC was invented by W. Wesley Peterson in 1961 [7]. The CRC treats the data bits as a binary polynomial with specific width, then it calculates the remainder from the division of data bits with a generator polynomial. the remainder of this division considered the checksum that should be attached with the data bits. At the receiver side, the data with the checksum divided by the same polynomial, if the result was zero that's confirm the data was received correctly [8].

Mathematically, a k-bit message can be considered as the coefficients of a polynomial  $B(x) = b_{k-1}x^{k-1} + \dots + b_1x^1 + b_0x^0$ . The most significant bit leads the data stream. Furthermore, an  $(m+1)$  bit generator polynomial  $P(x) = x^m + p_{m-1}x^{m-1} + \dots + p_1x^1 + p_0x^0$  of order  $m$  is selected. Calculations are performed in modulo-2 arithmetic. The CRC is the remainder of the division of  $x^m B(x)$  by  $P(x)$  and will be appended to the message [4].

CRC is well suited for the detection of burst errors which is common in the communication channels. The CRC called n-bit CRC when its value is n bits long. For example, it is called CRC-1 when it is used in the famous check type that called parity check and in this type there is only one bit attached to the original message, another example in CRC-4 there are 4 bits attached to the message and so on. In this paper, we are focusing on the CRC32 which is the type that used in the Ethernet frame.

CRC32 has a fixed polynomial value which is 0x04C11DB7. The transmitted message consists of the original message in addition to the reversed-byte swapped CRC for example, if the message has the hex value (31 32 33 34 35) the Ethernet CRC32 will be 0xCBF53A1C, and the transmitted message will be (31 32 33 34 35 1C 3A F5 CB). In the Ethernet frame the FCS (Frame Check Sequence) field is the CRC. The message for which the CRC32 calculated consists of the source and destination MAC, Ethernet frame length, in addition to the data, the Ethernet frame illustrated in Fig 1.

| Preamble | SFD | Destination | Source | Length | Data and Pad | FCS |
|----------|-----|-------------|--------|--------|--------------|-----|
| 7        | 1   | 6           | 6      | 2      | 46 – 1500    | 4   |

Fig. 1. IEEE 802.3 Ethernet Frame format

### III. PROPOSED WORK

The main aim of this paper is to increase the throughput of the CRC32 algorithm by performing parallel operations using the FPGA. The computation of the CRC32 performed by the lookup table method. In this method the possible values of the CRC32 pre-computed and saved in lookup tables. This pre-computed values called during the computation of the final checksum value rather than compute it every time. Each byte has its own CRC32 value that will be added as an argument in logical operations that can find the final result. The algorithm could be implemented using one lookup table, but to increase the efficiency of the system more than one lookup tables implemented simultaneously combined with another technique called slicing by N. The system designed based on FPGA Virtex-7, the related code written by Very High Speed Integrated Circuit Description Language (VHDL). And synthesized by Xilinx synthesis tool. After that the simulation performed on ISim which is Xilinx simulation tool and the simulation code written by the VHDL as well.

Accordingly, the precomputed CRC32 values must be prepared during the initialization of the system, Fig 2 show that a PC connected to FPGA, the role of this PC in the design is the generation of the pre-computed values of the lookup tables through Java program. After these values generated it will be transferred to the FPGA and saved inside RAMs, these RAMs designed inside the FPGA to reduce the transfer time, the design area and the power. This PC can be considered a microprocessor located outside the FPGA or inside it in the real implementation, but for simulation purposes the CRC32 values generated through software written in Java.

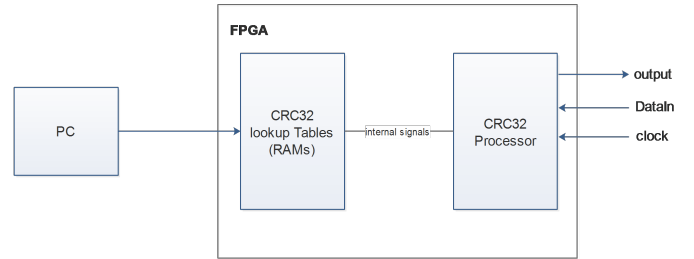


Fig. 2. System's block diagram

#### A. Generation of lookup tables

As mentioned earlier, lookup tables methodology can reduce the number of needed operations to find out the CRC32 for a stream of bytes. as long as the divisor (polynomial) is always fixed we can pre-compute the division for each possible byte and save the remainder in a lookup table. This can be performed through the below code

```

for (int dividend = 0; dividend <= 255; dividend++)
{
    int temp = dividend;
    for (int bits=0; bits<8; bits++) // generate the CRC
        temp = (temp >>1) ^ ((temp &1) * 0x04C11DB7);
    crcTable[dividend] = temp;
}

```

In this code, the dividend which includes the possible values for the bytes (from 0x00 to 0xFF) saved in the temp variable in each iteration. Then the temp variable processed in term of bits inside the second for loop. the equation behind the second for loop checkup the LSB of the temp, if it is '1' then the shifted right value of the temp will perform xor with the polynomial, this operation continues for the eight bits to find out the remainder of (dividend mod polynomial). This remainder saved in the lookup table Array that indexed by the dividend. Fig 3 shows the flowchart of lookup table generation. Now from this lookup table, we can find the CRC32 for only one byte in the period. For the purpose of parallelism another 15 tables generated, they are generated based on the values on this table. Next section will explain this point more.

### B. Slicing by 16

The design going to find out the CRC32 for 16 bytes (128 bit) in one operation that is mean 15 additional lookup tables should be generated as well. The generation of these tables depends on the first table that already generated, the below code responsible for this issue. the value for each byte on each lookup table depends on the preceding lookup table, this preceding value shifted to the right by 8 bit and then xor with lookup table 0 that indexed by the LSB byte from the preceding lookup table.

```
for(inti=0;i<=255;i++)
{
crcTable1[i] =
(crcTable0[i]>>8) ^ crcTable0[(int)(crcTable0[i]&0xFF)];

crcTable2[i] =
(crcTable1[i]>>8) ^ crcTable0[(int)(crcTable1[i] & 0xFF)];
...

crcTable15[i]=
(crcTable14[i]>>8) ^ crcTable0[(int)(crcTable14[i]& 0xFF)];
}
```

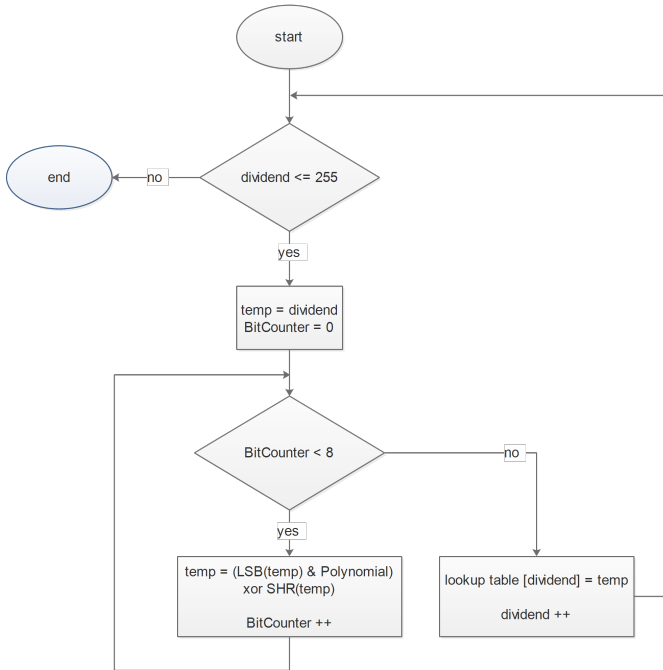


Fig. 3. flow chart for lookup table generation

### C. Hardware design for lookup tables

The implementation of lookup tables RAMs will be performed inside the FPGA that is in order to reduce the design area, decrease the power consumption and to increase the transferring speed between the FPGA and the RAMs. The sixteen RAMs created by VHDL code. A new type called lookUpTable defined as array consist of 256 locations, each location has 28 bit. Then sixteen signal of this type has defined. During the initialization, these memory locations loaded with the precomputed values.

### D. Implementation of Slicing-by-16 CRC32

In VHDL the entity is the description of the ports of the circuit. In this project, the entity consists of the ports that illustrated in Fig 4.

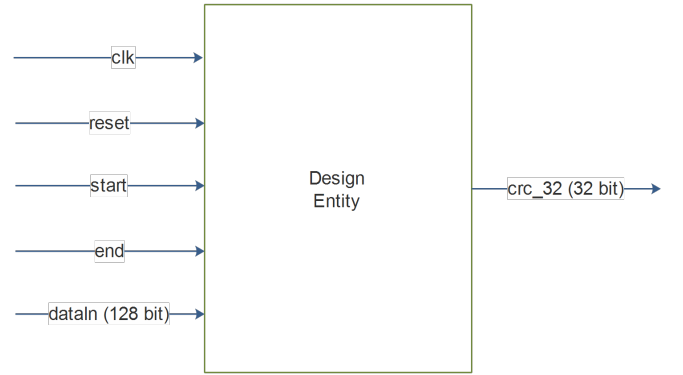


Fig. 4. Design Entity

TABLE I. VHDL ENTITY PORTS DESCRIPTION

| Port Name | Description                                                                                                                    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------|
| clk       | 1 bit input port, through it the circuit can receive its clock.                                                                |
| reset     | 1 bit input port used to reset the circuit and the CRC32 register.                                                             |
| start     | 1 bit input port, used to trigger the circuit of CRC32 computation.                                                            |
| end       | 1 bit input port, indicate the Ethernet frame reached the last bit and accordingly the computation of the CRC32 can be closed. |
| dataIn    | 128-bit port, should be connected to internal buffer. This port includes the data that needs CRC32 calculation.                |
| crc_32    | 32 bit output port, hold the result of CRC32 computation.                                                                      |

As illustrated in Fig 5, the stream of the data saved in a buffer. Splitting circuit slices the data into 12 bytes and 4 bytes. Each byte of the twelve will be used to address the RAMs (from RAM0 to RAM11), the remaining 32 bits XORed with the previous CRC32 value and then sliced into four bytes, each one used to address the RAMs (from RAM12 to RAM15). The addressed values in the RAMs XORed together to get the new value for the CRC32. There is no considerable time to get the output from the xor gates, so the sixteen RAMs could be accessed in the same time. The implementation of the code in the VHDL performed concurrently, that is will support to achieve the operation of this algorithm in parallel.

### E. Simulation

Isim simulator version 14.5 used to find out the result of the design, Isim uses the VHDL to write the simulation code. However, Isim creates a component for the unit under test. The component has the same ports as the design entity in the main design. furthermore, Isim automatically generates signals corresponds to the entity ports, these signals used as a stimulus for the unit under test. Regarding the clock, the Isim automatically also generate constant time identify the period of the pulses. The constant time by default generated to be 10 ns but based on the design requirements the time modified to 1.25 ns which provide a frequency equal to 800 MHz. the simulation has two process working concurrently, the first process generates the clock pulses and the second process responsible for the simulation stimulus. Inside the process of the stimulus 128 bit randomly assigned to the DataIn signal in a period specified in simulation code, behind that the control signals such as start, end and reset also considered in the simulation code. Fig 6 illustrates how the control signals and the dataIn stimulated in the simulation.

## IV. RESULTS

The simulation of this design performed based on virtex-7 FPGA with 800 MHz clock oscillator, this can lead to process 128-bits of the data in 1.250 ns. That is mean the throughput of the CRC32 can reach up to 102.4 Gbps which is the targeted result, this throughput is higher than the throughputs that mentioned in the previous works. Regarding the analysis of the operations that performed in the time domain of the clock pulses, in Fig 5 the highlighted combinational circuit responsible for performing xor operation between the previous result and the dataIn least significant bits. This circuit performed outside the process and independent of the clock pulses. During the falling edge of the clock pulses the buffer response to sending the data to the entity in the dataIn port. The time between the falling edge and the rising edge utilized to find out the output of the highlighted circuit in Fig 5.

Nevertheless, to find out the final result the none-highlighted circuit needs to work, this circuit restricted to work only during the rising edge of the clock pulses. Accordingly, the none-highlighted circuit added to process with the related sensitivity list in VHDL code in order to perform its work during the high pulse. Fig 6 illustrates the relationship between the dataIn, the output of the CRC32 and the related pulse.

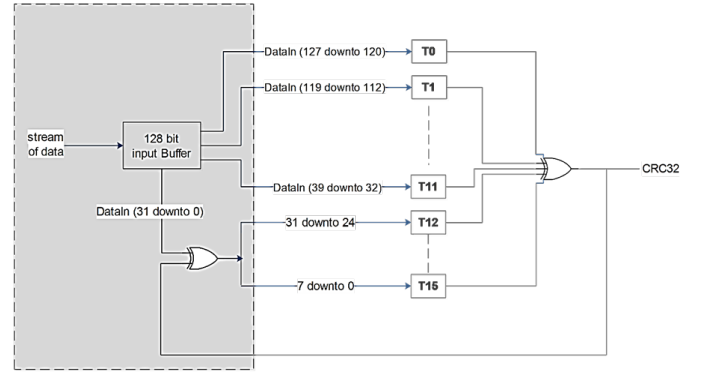


Fig. 5. CRC32 using slicing-by-16

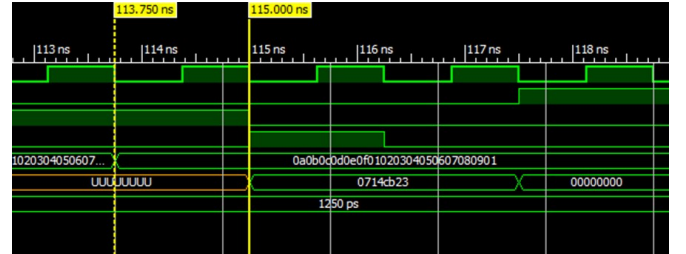


Fig. 6. The relation between the needed clock, dataIn (5<sup>th</sup> row) and the CRC32 output in the 6<sup>th</sup> row

the utilized resources from the FPGA which could be considered very low, which allow the system designer to merge it inside any system. The number of slices register that used is 27 from 408000, the number of slice LUTs is 550 from 204000. Table II illustrates device utilization summary that generated by Xilinx ISE.

On another hand, the total of the consumed power is 0.158W, the dynamic power consumption is only 0.015W. so most of the power consumption generated from the leakage or the quiescent which is around 0.143W. accordingly the power

consumption is low and could be considered very low when compared to the previous work.

TABLE II. DEVICE UTILIZATION SUMMARY

| Device Utilization Summary (estimated values) |      |           |             |
|-----------------------------------------------|------|-----------|-------------|
| Logic Utilization                             | Used | Available | Utilization |
| Number of Slice Registers                     | 27   | 408000    | 0%          |
| Number of Slice LUTs                          | 550  | 204000    | 0%          |
| Number of fully used LUT-FF pairs             | 27   | 550       | 4%          |
| Number of bonded IOBs                         | 164  | 600       | 27%         |
| Number of BUF6/BUF6CTRL/BUFHCES               | 2    | 200       | 1%          |

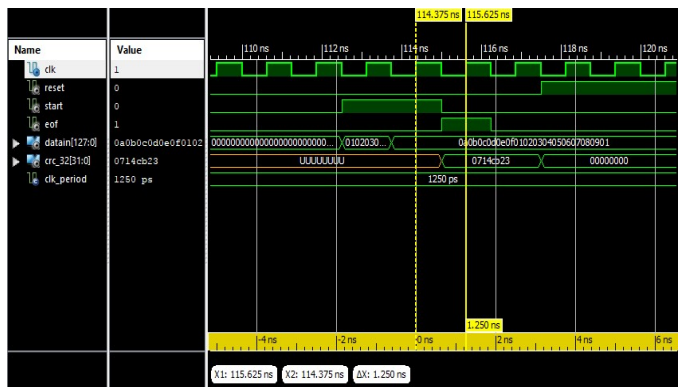


Fig. 7. After 1.250 ns since the dataIn provide the input, the CRC32 calculated

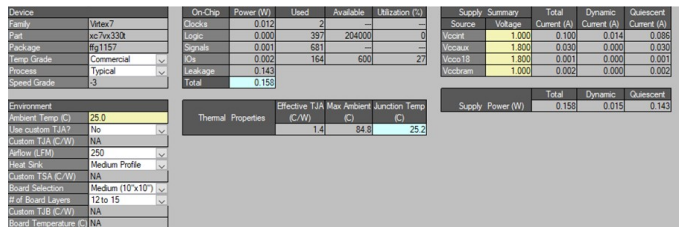


Fig. 8. Power consumption

## V. CONCLUSION

The demand on the data communication became very high, next year applications, the internet of things and machine to machine communications can affect the trend of the data consumption either from the user point of view or beside the companies, enterprises and data centers. This change affects directly on the networking devices in term of switching capability, processing, and finally the computation of the CRC32 for the error detection and correction which is

the main point that discussed in this project. The project aims to improve the processing capability of the CRC32 in order to meet the new requirements and speed. The targeted throughput is 100 Gbps and in order to achieve it, the design built based on the FPGA. The FPGA selected due to its concurrent behavior, it can perform more than one operation concurrently. Behind that, more than one algorithm has studied and the slicing by 16 algorithm selected, this algorithm reduces the computation of the CRC32 by creating lookup tables hold all the possible values of the CRC32. The code of the design and the simulation written by VHDL. The simulation test shows that; the achieved throughput is equal to 102.4 Gbps.

## REFERENCES

- [1] Cisco, "The Zettabyte Era: Trends and Analysis," 2016.
- [2] Sarwate, D.V., "Computation of Cyclic Redundancy Checks via Table Lookup," Comm. ACM, vol. 31, no. 8, pp. 1008-1013, Aug. 1988.
- [3] P. A. Anand and D. Bajarangbali, "Design of High Speed CRC Algorithm for Ethernet on FPGA Using Reduced Lookup Table Algorithm," presented at the IEEE Annual India Conference (INDICON), India 2016.
- [4] M. Braun, J. Friedrich, T. Grün, and J. Lember, "Parallel CRC computation in FPGAs," in Proc. 6th Int. Workshop Field-Program. Logic, Smart Appl., New Paradigms Compilers (FPL), London, U.K., 1996, pp. 156-165, Springer-Verlag.
- [5] C. Cheng and K. Parhi, "High-speed parallel CRC implementation based on unfolding, pipelining, and retiming," IEEE Trans. Circuits Syst. II, Expr. Briefs, vol. 53, no. 10, pp. 1017-1021, Oct. 2006.
- [6] C. Kennedy and A. Reyhani-Masoleh, "High-speed parallel CRC circuits," in Proc. 42nd Asilomar Conf. Signals, Syst. Comput., Oct. 2008, pp. 1823-1829.
- [7] W. Peterson and D. Brown, "Cyclic codes for error detection," Proc. IRE, vol. 49, no. 1, pp. 228-235, 1961.
- [8] T. Ramabadran and S. Gaitonde, "A tutorial on CRC computations," IEEE Micro, vol. 8, no. 4, pp. 62-75, Aug. 1988.